

STATS AND CONTROL 2026

May 2026

A Practical Reinforcement Learning Control Framework for Complex Process Systems

A. H. Hamed

Supervisor: P. Mhaskar

Department of Chemical Engineering, McMaster University

In collaboration with Linde plc.
Kingston University

Central question

How can reinforcement learning improve constrained chemical process control while keeping the controller practical, interpretable, and safe?

- ① Motivation: why fixed linear MPC and naive online RL each face practical limits
- ② Problem settings: the polymer reactor benchmark and the Aspen Dynamics C_2 splitter
- ③ **Project 1** (published): MPC-pretrained RL with offset-aware online adaptation
- ④ **Project 2** (ongoing): RL-assisted MPC as a supervisory adaptive layer
- ⑤ **Project 3** (ongoing): Lyapunov-filtered safe RL with MPC fallback
- ⑥ **Closing**: Main takeaways, current limitations, and next steps

MPC works through its model, and the model is the bottleneck

Why MPC is the right baseline

- MPC uses an internal process model to predict future behavior and optimize inputs under constraints (**Rawlings et al., 2017**).
- That model lets MPC coordinate multivariable decisions online.

Why the model becomes the bottleneck

- **First-principles models:** mechanistic, but expensive to build and hard to maintain for large changing plants (**Daoutidis et al., 2023**).
- **Data-driven models:** flexible, but they need informative data, monitoring, and revalidation (**Hassanpour et al., 2020**).

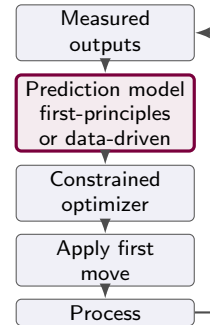


Figure: MPC relies on a prediction model inside the closed loop, so controller quality tracks model quality.

Takeaway: MPC performance depends directly on model quality and model upkeep.

Industry uses linear MPC because it is practical, but validity is local

Why linear MPC remains common

- Linear MPC remains a practical industrial choice for constrained multivariable process control (**Darby et al., 2012**).
- Linear empirical models are easier to identify, validate, and maintain than large nonlinear models.
- The resulting online optimization problem is tractable and well understood.

Why linear MPC still faces problems

- One linear model is usually most accurate near the operating region where it was identified.
- Nonlinearity, plant drift, and disturbances can create prediction error, offset, and retuning needs.

Takeaway: Linear MPC is practical and useful, but it becomes a local controller when plant behavior moves outside the identification region.

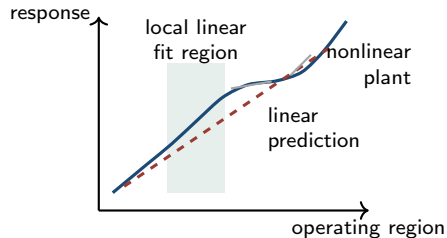


Figure: a linear model can match well near its identification region, then diverge as the plant moves away from nominal operation.

RL learns a policy from interaction and long-horizon reward

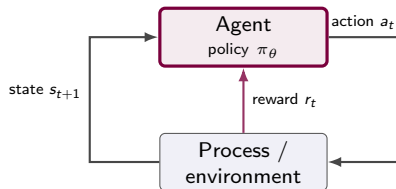


Figure: RL updates a policy from interaction data and long-horizon reward.

What the controller learns

The agent maps state to action, $a_t = \pi_\theta(s_t)$, and learns from transition data (s_t, a_t, r_t, s_{t+1}) . The policy is improved by increasing

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \right].$$

Why this is attractive for control

- Once trained, the policy can provide fast nonlinear feedback.
- RL is attractive for nonlinear or time-varying plants, but the training data and reward design become critical (Nian et al., 2020).

Takeaway: RL can learn long-horizon feedback, but process-control RL still needs carefully designed data and rewards.

Those RL assumptions break in industrial process control

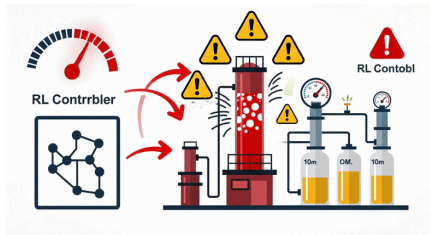


Figure: exploration that is acceptable in simulation can create unsafe or off-spec transients on a real plant.

Why direct online RL is difficult

- Process dynamics are slow, so data arrive in real operating time.
- Exploration can violate constraints, create off-spec product, and stress actuators.
- Plants cannot be reset after each poor trajectory, and delayed consequences make credit assignment harder.

Implication for this talk

- Direct online RL is not a realistic industrial baseline.
- Useful RL needs control structure, safer initialization, and a constrained role (**Nian et al., 2020**).

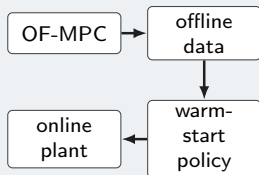
Takeaway: The practical question is not how to explore freely online, but how to make RL learn within a constrained control structure.

There is a practical place for RL if it respects control structure

Transition

The question is therefore not whether RL should replace process control, but where learning can help while keeping the controller practical, interpretable, and safe.

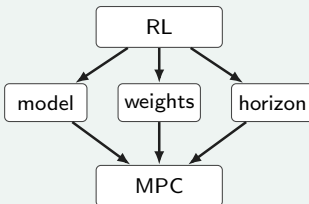
1. MPC-pretrained RL



Role of RL

RL starts from an MPC-like policy instead of unsafe random exploration.

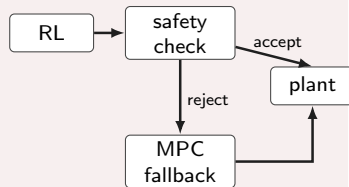
2. RL-assisted MPC



Role of RL

RL helps MPC adapt without discarding the MPC optimizer.

3. Safe RL with fallback



Role of RL

RL is used only when the candidate action passes a model-based safety test.

The Aspen Dynamics C_2 splitter is the motivating industrial problem

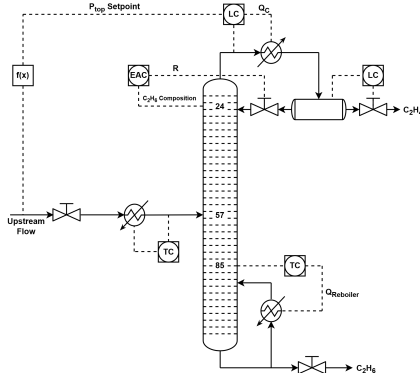


Figure: process diagram of the ethylene splitter used as the industrial-scale case study.

Why this case study matters

- The C_2 splitter is a high-purity ethylene separation with more than 100 trays, strong coupling, and expensive dynamics (**Salerno et al., 2011**).
- The control objective is tracking tray-24 ethane impurity x_{24, C_2H_6} and tray-85 temperature T_{85} using reflux flow and reboiler duty.

Why it is useful for this talk

- Aspen Dynamics is interfaced with Python, so the controller is tested on a high-fidelity closed loop rather than a toy simulator.
- A full closed-loop scenario is computationally expensive, so this case is used as the industrial-scale validation problem rather than the rapid method-development platform.

The polymer reactor is the illustrative benchmark for method development

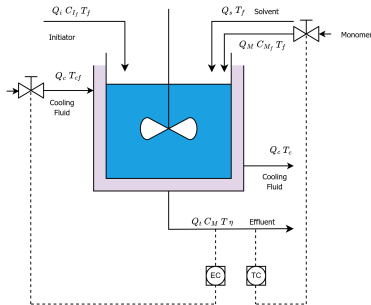


Figure: styrene polymerization reactor used as the nonlinear MIMO benchmark in this work.

Why this case study matters

- The reactor is nonlinear, strongly coupled, and constrained, so a fixed linear controller is only an approximation (**Bustos et al., 2016**).
- Manipulated inputs are the monomer flow rate Q_M and coolant flow rate Q_C .
- Controlled outputs are intrinsic viscosity η and reactor temperature T .

Why it is useful for this talk

- In our current workflow, one closed-loop run is practical enough for repeated controller development, roughly on the order of one hour.
- Because closed-loop runs are much faster than the Aspen C₂ case, the polymer reactor is used to debug learning behavior, reward design, and ablation studies before transfer to the industrial-scale system.

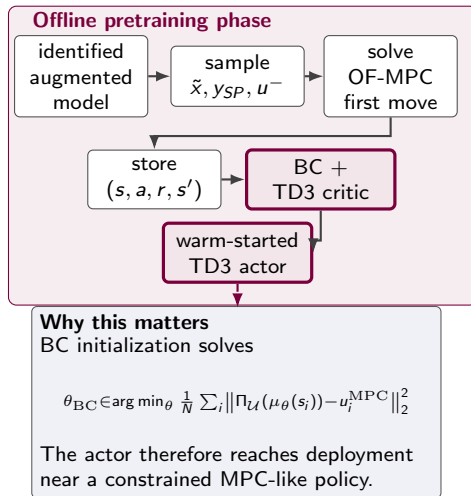
Project 1 starts from OF-MPC, so the actor is useful before online RL begins

Practical idea

Use the controller already trusted in practice to create the initial RL policy, instead of beginning online learning from unsafe random actions.

Offline transition generated from OF-MPC

$$s_t = \begin{bmatrix} \tilde{x}_t \\ y_{SP} \\ u_{t-1} \end{bmatrix}, \quad a_t = u_t^{\text{MPC}}, \quad s_{t+1} = \begin{bmatrix} \tilde{x}_{t+1} \\ y_{SP} \\ u_t^{\text{MPC}} \end{bmatrix}$$
$$r_t = -\left(\|y_{t+1} - y_{SP}\|_Q^2 + \|\Delta u_t\|_R^2\right)$$



RL1 improves settling, but still leaves a small offset

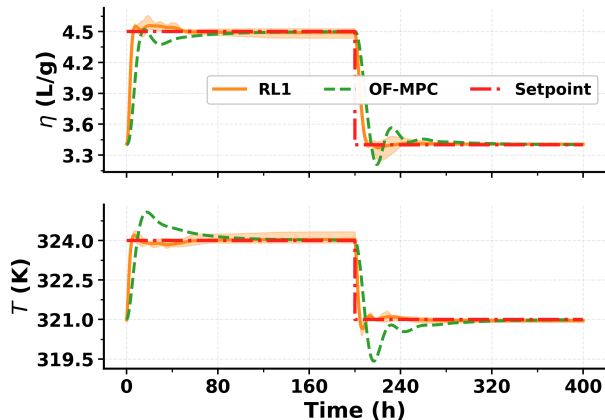


Figure: RL1 settles faster than OF-MPC under operating changes, but a small final offset remains.

RL1

Warm-started TD3 actor with the original one-step MPC-style reward

$$r_t^{\text{RL1}} = -(\|e_{t+1}\|_Q^2 + \|\Delta u_t\|_R^2)$$

Main observation

- Faster settling than OF-MPC
- Stable under drift
- Small residual offset remains

Takeaway: Pretraining fixes initialization, but RL1 still under-emphasizes small final corrections near the setpoint.

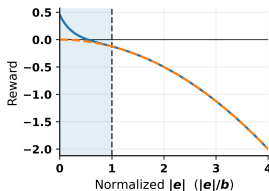
RL2 changes the online learning signal, not the controller

1. Offset-aware reward shaping

RL1 uses the one-step quadratic reward. RL2 increases sensitivity near the target using

$$z_i = \frac{|e_i|}{b_i}$$

so small residual offsets remain visible to TD3.



2. Mixed replay for adaptation

Each batch mixes

$$\mathcal{B}_{\text{PER}} \cup \mathcal{B}_{\text{recent}} \cup \mathcal{B}_{\text{uniform}}$$

so updates see informative errors, fresh drift data, and broad coverage.

Same TD3 backbone

$$s_t = \begin{bmatrix} \hat{x}_t \\ y_{SP} \\ u_{t-1} \end{bmatrix}, \quad u_t = \Pi_{\mathcal{U}}(\mu_{\theta}(s_t))$$

RL1 versus RL2

RL1	quadratic reward + standard replay
RL2	near-target reward + mixed replay
Same	warm start, observer, projection, TD3

Key point: RL2 targets residual offset by changing what TD3 learns from, while keeping the same bounded actor.

On the polymer reactor, RL2 keeps the transient gain and reduces offset

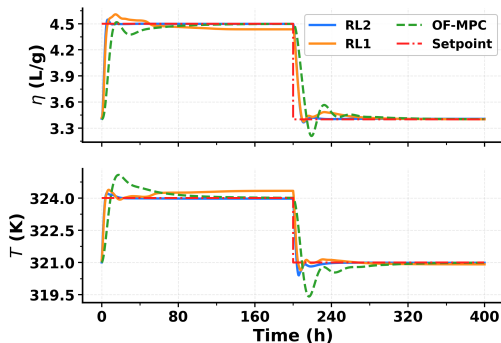
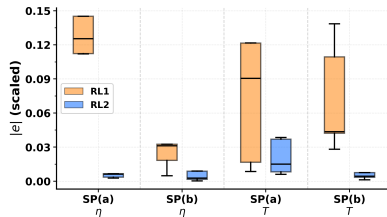


Figure: evaluation under unseen operating changes. RL2 settles faster than OF-MPC and finishes closest to the setpoints.

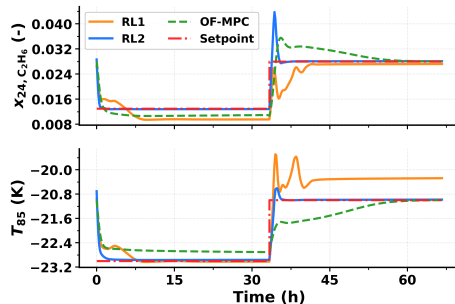
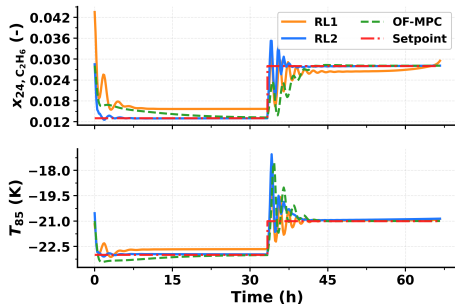


Error reduction vs RL1

	SP(a)	SP(b)
η	92%	76%
T	84%	90%

Averaged over 5 runs, RL2 gives smaller medians and tighter spreads than RL1.

On the Aspen C_2 splitter, RL2 remains better in both scenarios



Cross-scenario message

Fluctuation

Ramp

Overall

RL2 improves disturbance rejection around the operating region.

RL2 tracks the moving target while keeping the offset advantage.

x_{24, C_2H_6} has roughly 90–99% lower steady-state error than RL1, with substantial improvement in T_{85} .

Project 1 takeaway: useful MPC-pretrained RL, but empirical safety

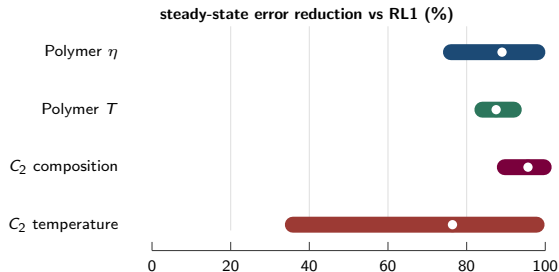


Figure: min-to-max range across reported setpoints and scenarios. White marker shows the mean.

Publication

Published in *Industrial & Engineering Chemistry Research (I&ECR)*.

Takeaway: Project 1 shows practical benefits, but the next step is to make learning more structured and safety-aware.

Established

- MPC pretraining avoids random online initialization.
- RL2 improves final offset without changing the plant-side architecture.
- The improvement transfers from polymer to the Aspen C_2 case.

Still empirical

- Safety is observed in simulation, not certified before each move.
- This motivates keeping MPC more explicitly in the loop.

Project 2: RL supervises MPC instead of replacing it

Core idea

- Project 2 keeps the same offset-free MPC backbone from Project 1.
- RL does not become the low-level controller; it only chooses supervisory MPC parameters online.
- The plant still receives the first move from the MPC optimizer, so constraints and receding-horizon structure stay intact.
- The learned supervisory variable can be the horizon, model, penalties, residual correction, or later a combined action.

What is intentionally reused

- The replay-buffer workflow is the same off-policy transition storage used in Project 1.
- The reward function is also the same Project 1 shaped reward, so the comparison isolates *what MPC knob is adapted*.
- The question is therefore structural: which

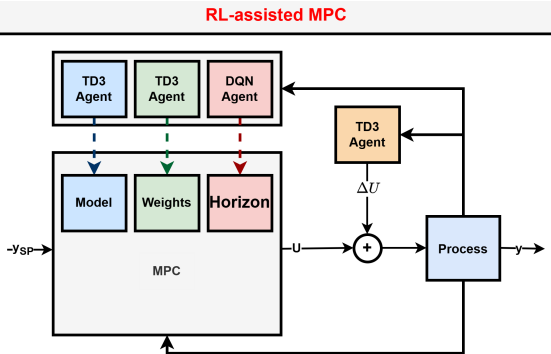


Figure: the policy updates supervisory MPC parameters while the optimizer still computes the applied move.

Project 2 method: shared state, shared reward, different supervisory actions

Shared RL state and reward

$$\bar{s}_k = [\hat{x}_k^{aug}, y_k^{sp}, u_{k-1}],$$

$$s_k = \begin{cases} \bar{s}_k, & \text{standard state,} \\ [\bar{s}_k, \nu_k, \varepsilon_k, \rho_k], & \text{mismatch state,} \end{cases}$$

$$\nu_k = T\left(\frac{y_k - \hat{y}_k}{\sigma_\nu}\right), \quad \varepsilon_k = T\left(\frac{y_k - y_k^{sp}}{\sigma_\varepsilon}\right).$$

$$r_k = r_{\text{Proj1}}(e_k, \Delta u_k, y_k^{sp}),$$

with the same Project 1 shaped reward and the same replay tuple

$$(s_k, a_k, r_k, s_{k+1}).$$

Shared rollout algorithm

- ➊ Update the observer and build s_k from the augmented estimate, setpoint, previous input, and mismatch features if enabled.
- ➋ During warm start use the nominal supervisory action; afterwards evaluate the RL policy every decision interval.
- ➌ Map the policy output into θ_k : horizons, model multipliers, weight multipliers, or residual correction.
- ➍ Solve MPC with θ_k and apply only the first move; for the residual method, correct that move before plant actuation.
- ➎ Compute r_k , append (s_k, a_k, r_k, s_{k+1}) to replay, and update the off-policy agent.

Supervisory action map

Horizon: $a_k \in \mathcal{H}$ $\theta_k = (N_{\nu, \nu}, N_{\varepsilon, \varepsilon})$

Code path used for this slide

- State construction: `utils/state_features.py`
- Reward construction: `utils/rewards.py`
- Shared `utils/push/train` logic:

Horizon Agent: RL chooses the prediction horizon instead of fixed look-ahead

Method

The horizon agent uses dueling DQN because the supervisory action is discrete:

$$a_k \in \mathcal{H} = \{(N_p, N_c)^{(1)}, \dots, (N_p, N_c)^{(m)}\}.$$

The network estimates

$$Q(s_k, a) = V(s_k) + A(s_k, a) - \frac{1}{|\mathcal{H}|} \sum_{a'} A(s_k, a'),$$

then MPC solves with the selected (N_p, N_c) .

Interpretation

- Shorter horizons react faster when the state is near a disturbance or setpoint switch.
- Longer horizons preserve anticipation when the process is close to nominal.
- This agent changes optimization look-



Reward across seeds: the horizon policy learns steadily and finishes well above the fixed-MPC reward line.



Last-episode evaluation: both viscosity and temperature settle closer to the setpoint with smaller late oscillation than fixed MPC.

Matrix Agent: RL perturbs the internal MPC model used for prediction

Method

The matrix agent keeps the optimizer fixed but changes the linear prediction model:

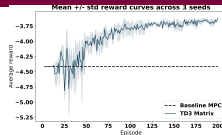
$$A_k = \alpha_k A_0, \quad B_k = B_0 \text{diag}(\beta_{1,k}, \beta_{2,k}).$$

TD3 outputs a continuous action $a_k = [\alpha_k, \beta_{1,k}, \beta_{2,k}]^\top$, and MPC computes the move using (A_k, B_k) .

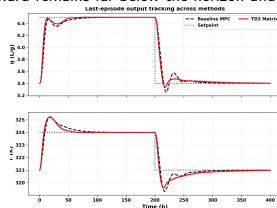
Interpretation

- α_k changes the effective process memory and speed.
- $\beta_{i,k}$ change the predicted input gains seen by MPC.
- This is powerful when model mismatch dominates, but it is also the easiest supervisory family to overtune.

Latest polymer result



Reward across seeds: learning improves from the early unstable region, but the converged reward remains far below the horizon and weight agents.



Last-episode evaluation: the matrix agent matches the main setpoint transitions reasonably well, but the aggregate advantage over baseline is limited in this run.

Weight Agent: RL retunes the MPC cost to trade off speed and smoothness online

Method

The weight agent leaves the model fixed and changes the objective:

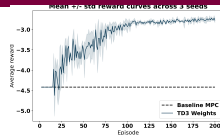
$$J_k = \sum_{i=1}^{N_p} \|y_{k+i} - y_{k+i}^{sp}\|_{Q_k}^2 + \sum_{i=0}^{N_c-1} \|\Delta u_{k+i}\|_{R_k}^2.$$

TD3 outputs positive multipliers such that

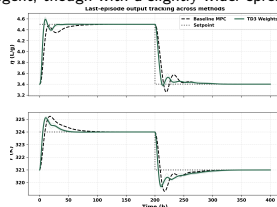
$$Q_k = \text{diag}(q_k)Q_0, \quad R_k = \text{diag}(r_k)R_0.$$

Interpretation

- Larger q_k make MPC chase output error more aggressively.
- Larger r_k penalize input movement and produce smoother actions.
- Because the prediction model is



Reward across seeds: the weight agent climbs steadily and stabilizes near the horizon agent, though with a slightly wider spread.



Last-episode evaluation: output tracking improves against baseline MPC while keeping the response smoother than aggressive model adaptation.

Residual Agent: RL adds a correction around the nominal MPC move

Method

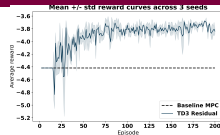
The residual agent preserves nominal MPC and learns only an additive correction:

$$u_k = u_k^{\text{MPC}} + \Delta u_k^{\text{res}},$$

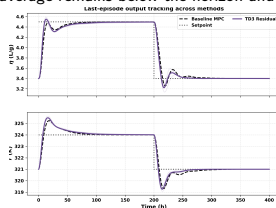
with $\Delta u_k^{\text{res}} = \pi_\phi(s_k)$ from TD3. The executed move is still clipped or projected to the admissible input set before it reaches the plant.

Interpretation

- MPC keeps the stabilizing nominal policy in the loop.
- RL is responsible only for local correction of mismatch or missed transient behavior.
- This family is attractive when we want adaptation without changing the internal optimization model.



Reward across seeds: the residual policy improves from its initial low-reward region, but the final average remains below the horizon and weight agents.



Last-episode evaluation: the residual action gives a good nominal transient, but the aggregate benefit is smaller than the best supervisory families in this notebook run.

Combined Agent: four RL supervisors coordinate one disturbance-mode MPC loop

Method

The combined supervisor runs four branch-specific agents around the same disturbance-mode MPC loop:

$$a_k = (a_k^H, a_k^M, a_k^W, a_k^R).$$

Their actions are mapped as

$$(N_{p,k}, N_{c,k}) = \mathcal{H}(a_k^H),$$

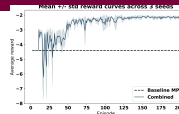
$$(A_k, B_k) = \mathcal{M}(a_k^M; A_0, B_0),$$

$$(Q_k, R_k) = \mathcal{W}(a_k^W; Q_0, R_0),$$

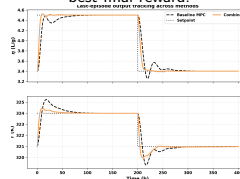
$$u_k = \Pi_{\mathcal{U}} \left(u_k^{MPC} + \rho_k \Delta u_k^{res}(a_k^R) \right).$$

Interpretation

- Horizon changes look-ahead.
- Matrix changes the internal model.
- Weights retune aggressiveness.
- Residual corrects the final move after MPC solves.



Reward across seeds, disturbance compare mode: the combined policy reaches the best final reward.



Last-episode evaluation vs disturbance MPC: both outputs improve while MPC remains central.

Disturbance-mode polymer summary versus the same MPC baseline

Baseline and counting rule

All metrics and reward comparisons use the same disturbance baseline

Polymer/Data/mpc_results_dist.pickle,

with $\overline{\text{RMSE}} = 0.3795$ and $\overline{\text{IAE}} = 13193.6$. 'Win episodes' counts episodes 2:200 where

$$\bar{R}_{ep}^{RL} > \bar{R}_{ep}^{MPC}$$

across the three saved seeds, so each method is scored out of 597.

Summary table

Method	Final reward	%Reward	ΔRMSE	ΔIAE	Win eps
Horizon	-2.60	+41.2%	+0.087	+5693.8	587/597
Matrix	-3.63	+17.8%	+0.006	-527.9	535/597
Weights	-2.77	+37.3%	+0.046	+2961.6	567/597
Residual	-3.84	+13.1%	+0.011	+686.6	545/597
Combined	-2.10	+52.5%	+0.101	+4809.0	559/597

Positive Δ values mean improvement over disturbance MPC. Combined is best on final reward, RMSE gain, and IAE reduction. Horizon wins the most episode-level reward comparisons against MPC.

Takeaway: The disturbance-mode study says two things at once: combined is the best final controller, while horizon is the most consistently better-than-MPC learner across episodes.